

Traduction des langages

TD5 génération de code

1 AST Placement

```
module AstPlacement =
struct

  (* Expressions existantes dans notre langage *)
  (* = expression de AstType *)
  type expression = AstType.expression

  (* instructions existantes dans notre langage *)
  type bloc = instruction list * int (* taille du bloc *)
  and instruction =
    | Declaration of Tds.info_ast * expression
    | Affectation of Tds.info_ast * expression
    | AffichageInt of expression
    | AffichageRat of expression
    | AffichageBool of expression
    | Conditionnelle of expression * bloc * bloc
    | TantQue of expression * bloc
    | Retour of expression * int * int (* taille du retour et taille des paramètres *)
    | Empty (* les nœuds ayant disparus: Const *)

  (* informations associées à l'identificateur (dont son nom), liste de paramètres, corps, expression de retour *)
  (* Plus besoin de la liste des paramètres mais on la garde pour les tests du placements mémoire *)
  type fonction = Fonction of Tds.info_ast * Tds.info_ast list * bloc

  (* Structure d'un programme dans notre langage *)
  type programme = Programme of fonction list * bloc

end
```

2 Signatures de la passe Génération de code

```
(* AstPlacement.expression -> string *)
let rec analyse_code_expression e = ...

(* AstPlacement.instruction -> string *)
let rec analyse_code_instruction i = ...
(* AstPlacement.bloc -> string *)
  and analyse_code_bloc (li, taille) = ...

(* AstPlacement.fonction -> string *)
let analyse_code_fonction (Fonction (info, _, (li, _))) = ...

(* AstPlacement.programme -> string *)
let analyser (Programme (fonctions, prog)) = ...
```

3 Squelette de analyse_code_expression

```
(* AstPlacement.expression -> string *)
```

```
let rec analyse_code_expression e =
```

```
  match e with
```

```
  | AppelFonction (info, le) ->
```

```
  | Ident info -> (
```

```
  | Booleen b ->
```

```
  | Entier i ->
```

```
  | Unaire (op, e1) -> (
```

```
  | Binaire (b, e1, e2) -> (
```

4 Squelette de analyse_code_instruction

```
(* AstPlacement.instruction -> string *)
let rec analyse_code_instruction i =
  match i with
  | Declaration (info, e) -> (

    | Affectation (info, e) -> (

    | AffichageInt e ->

    | AffichageRat e ->

    | AffichageBool e ->

    | Conditionnelle (c, t, e) ->

    | TantQue (c, b) ->

    | Retour (e, tailleRet, tailleParam) ->

    | Empty ->

  and
  (* AstPlacement.bloc -> string *)
  and analyse_code_bloc (li, taille) =
```

5 Squelette de analyser et analyse_type_fonction

```
(* AstPlacement.fonction -> string *)  
let analyse_code_fonction (Fonction (info, _, (li, _))) =
```

```
(* AstPlacement.programme -> string *)  
let analyser (Programme (fonctions, prog)) =
```
